WAD - CheatSheet

WEB APPLICATION DEVELOPMENT | BE05000281 | Sem 5 | GTU

New Syllabus 2024-25 | Old Papers Analyzed: 13 (S2016 – W2025)

> **Compiled BY : Manthan A. D. [in]**

SUBJECT OVERVIEW

Ch.	CH. Name	Content
1	**Web technologies**	Client-server, static/dynamic, frontend/backend
2	**HTML + CSS**	Structure, tables, forms, responsive layout
3	**JavaScript**	Variables, DOM/events, logic, async
4	**APIs + HTTP**	Methods, CRUD, JSON, status codes
5	**Node.js**	Express, APIs, database
6	**React.js**	JSX, hooks, props/state
7	**Deployment**	Hosting, DNS, SEO

CH. 1 . INTRO. TO WEB TECHNOLOGIES

Official syllabus: web technologies, evolution of websites, client-server architecture, frontend/backend/database, banking and finance, static vs dynamic websites.

Important terms

- **WWW:** World Wide Web. Interconnected web resources accessed over the Internet.
- **HTTP:** Hypertext Transfer Protocol. Protocol for web request-response communication.
- **Client:** Software/device requesting a resource, commonly a browser.
- **Server:** System that receives requests and provides responses/services.
- **Frontend:** User-facing part of an application.
- **Backend:** Server-side logic and processing.
- **Database:** Organized collection of data for storage and retrieval.
- **Static website:** Serves predefined content; it does not generate personalized content from server-side logic for each request.
- **Dynamic website:** Content or responses can change using data, user input or application logic.

Client-server architecture . IMP

```
Client / Browser
       |
       | HTTP request
       v
   Web server / API
       |
       | Query (if needed)
       v
    Database
       |
       | Result
       v
   Web server / API
       |
       | HTTP response
       v
Client / Browser
```

Draw it in the exam: Client → Server → Database → Server → Client. Label request and response.

Remember: Client requests. Server processes. Database stores/retrieves data when needed. Server returns a response.

Client-side vs server-side scripting . M.IMP

Client-side	Server-side
Executes primarily in the browser	Executes on the server
HTML, CSS and JavaScript are common technologies	Node.js with Express.js is one example
Handles UI interaction and basic validation	Handles business logic, database operations and authorization
Code delivered to browser can be inspected	Server implementation can remain private
Cannot securely enforce server permissions by itself	Must enforce permissions and validate untrusted input

Banking and finance

- **Frontend:** Account dashboard, forms, transaction UI.
- **Backend:** Transaction processing, authentication and business rules.
- **Database:** Account records and transaction history.
- **Security:** HTTPS, server-side authorization and secure sessions.

Static vs dynamic websites

```
STATIC
Browser --> HTML/CSS --> Predefined page

DYNAMIC
Browser --> Server --> Application logic
                          |
                          v
                       Database
                          |
```

```
                          v
                  Generated response
```

Keywords: Client-server, request-response, frontend, backend, static vs dynamic, client-side vs server-side.

CH. 2 . HTML, CSS AND BOOTSTRAP

Highest-priority topics in the PYQ analysis: CSS types, HTML tables, forms, positioning, Flexbox and Bootstrap.

Important terms

- **HTML:** HyperText Markup Language. Defines webpage structure.
- **CSS:** Cascading Style Sheets. Controls appearance and layout.
- **Tag:** Markup such as `<p>` or `<h1>`.
- **Element:** The complete structure, commonly including opening tag, content and closing tag.
- **Attribute:** Additional information inside a tag, such as `href`, `id` or `required`.
- **Semantic HTML:** Elements describing their meaning, such as `<header>`, `<main>` and `<footer>`.
- **Responsive design:** Layout adapts to screen sizes and devices.
- **Bootstrap:** Frontend framework with CSS utilities, responsive grid and UI components.

Basic HTML skeleton . Essential

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
```

```
  <title>My Page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hello, World!</h1>
  <p>My first webpage.</p>
  <script src="script.js" defer></script>
</body>
</html>
```

- `<!DOCTYPE html>`: Declares HTML5.
- `<head>`: Contains metadata, title and resource links.
- `<body>`: Contains page content.
- `defer`: Downloads script while parsing HTML and executes it after parsing.

Basic tags

Purpose	Elements
Headings and text	`<h1>` to `<h6>`, `<p>`, `<strong>`, `<em>`
Links and images	`<a href="">`, `<img src="" alt="">`
Grouping	`<div>`, `<span>`
Semantic structure	`<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`
Lists	`<ul>`, `<ol>`, `<li>`
Table	`<table>`, `<tr>`, `<th>`, `<td>`
Form	`<form>`, `<label>`, `<input>`, `<select>`, `<textarea>`, `<button>`

HTML tables . M.IMP

- `rowspan`: A cell spans multiple rows.
- `colspan`: A cell spans multiple columns.
- `th`: Header cell.
- `td`: Data cell.

```html
<table border="1">
  <tr>
    <th rowspan="2">Name</th>
    <th colspan="2">Marks</th>
  </tr>
  <tr>
    <th>WAD</th>
    <th>CN</th>
  </tr>
  <tr>
    <td>Manthan</td>
    <td>85</td>
    <td>90</td>
  </tr>
</table>
```

Table layout

Name\Marks	WAD	CN
Manthan	85	90

Name spans two rows. Marks spans two columns in the first row.

Visual replacement . rowspan and colspan

```
+----------+----------------------+
| Name     |        Marks         |
| (2 rows) +----------+-----------+
|          |   WAD    |    CN     |
+----------+----------+-----------+
| Manthan  |    85    |    90     |
+----------+----------+-----------+
```

`rowspan="2"` makes the Name cell occupy two rows. `colspan="2"` makes Marks occupy two columns.

Method: Count the final number of columns, then place each cell. Do not add another cell where a `rowspan` or `colspan` already occupies space.

HTML forms and attributes . M.IMP

Control	Purpose
`text`	Single-line text
`email`	Email input with browser-level validation
`password`	Masks entered text
`radio`	Choose one option from a group
`checkbox`	Choose independent options
`select`	Dropdown list
`textarea`	Multiline text
`submit`	Submits a form

Essential attributes: `id` (identifier), `name` (submitted field name), `value` (control value), `required` (required input), `placeholder` (hint), `type` (input behaviour), `action` and `method` (form destination and method).

```
<form>
  <label for="email">Email</label>
  <input id="email" name="email"
         type="email" required>

  <p>Year</p>
  <label>
    <input type="radio" name="year" value="1">
    First
  </label>

  <label>
    <input type="checkbox" name="agree" required>
    Agree to terms
  </label>
```

```
  <button type="submit">Submit</button>
</form>
```

Radio controls share the same `name` to form a single-choice group. Browser validation is useful, but the backend must validate submitted data too.

Types of CSS . M.IMP

Type	Example	Scope
Inline	`<p style="color:red">Text</p>`	One element
Internal/embedded	CSS inside `<style>`	One HTML document
External	`<link rel="stylesheet" href="style.css">`	Reusable stylesheet

Class vs ID

```
.card { color: white; }  /* Class: reusable */
#header { color: blue; } /* ID: intended unique element */
```

- Class selector: `.name`
- ID selector: `#name`
- Element selector: `p`
- Universal selector: `*`
- Descendant selector: `.card p`

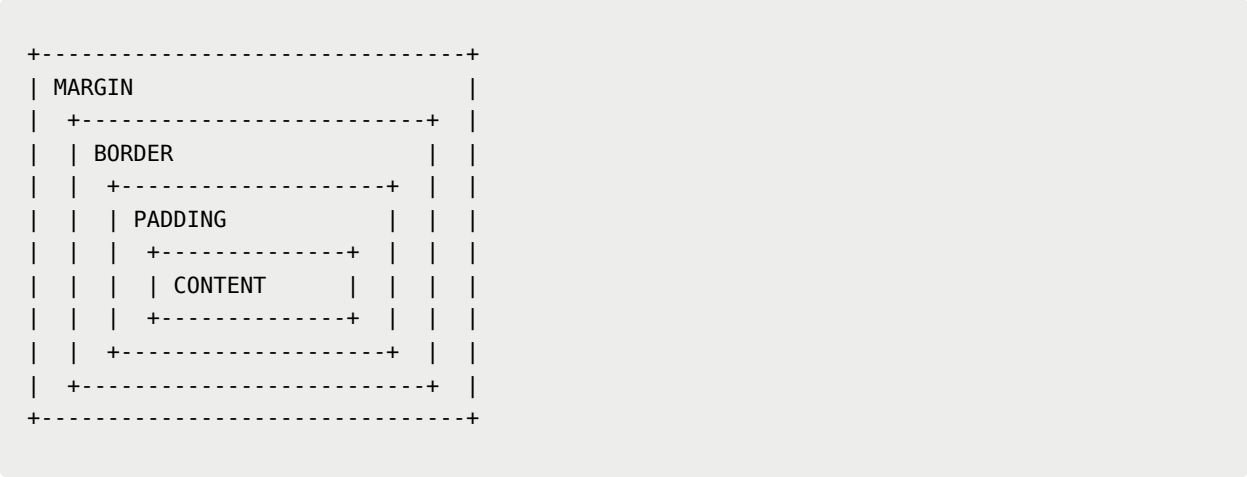
Specificity reminder: IDs generally outrank classes, and classes outrank element selectors. The cascade also considers importance, origin and source order.

CSS box model

CSS box model . outside to inside

Layer	Meaning
Margin	Space outside the border
Border	Boundary around the element

Layer	Meaning
Padding	Space inside the border
Content	Text, image or other content



Remember: Content → Padding → Border → Margin.

- **Content:** Text, image or other content.
- **Padding:** Space inside the border.
- **Border:** Boundary around the element.
- **Margin:** Space outside the border.

Positioning and Flexbox . IMP

Property	Meaning
`static`	Default position in normal flow
`relative`	Stays in normal flow; offsets from its normal position
`absolute`	Removed from normal flow; positioned relative to a containing block
`fixed`	Positioned relative to the viewport in typical cases
`sticky`	Switches between normal and stuck positioning
`z-index`	Controls stacking order within applicable stacking contexts

```
.container {
  display: flex;
```

```
  justify-content: space-between;
  align-items: center;
  gap: 16px;
  flex-wrap: wrap;
}
```

- `justify-content`: Main-axis alignment.
- `align-items`: Cross-axis alignment.
- `flex-direction`: Row or column.
- `flex-wrap`: Whether items can wrap.

Responsive design and Bootstrap

Media query:

```
.cards {
  display: flex;
  gap: 12px;
}

@media (max-width: 600px) {
  .cards {
    flex-direction: column;
  }
}
```

Bootstrap grid: a 12-column responsive system.

```
<div class="container">
  <div class="row">
    <div class="col-12 col-md-6">Left</div>
    <div class="col-12 col-md-6">Right</div>
  </div>
</div>
```

- On small screens, each section spans 12 columns.
- At the medium breakpoint and above, each section spans 6 columns.
- Breakpoint names and values depend on the Bootstrap version.

Keywords: HTML skeleton, semantic tags, `rowspan`, `colspan`, forms, CSS types, class vs ID, box model, positioning, Flexbox, media query, 12-column grid.

CH. 3 . JAVASCRIPT FUNDAMENTALS

M.IMP: Regex form validation, DOM manipulation, event handling and logic programs. Also revise callbacks, promises, asynchronous execution and localStorage.

Important terms

- **JavaScript (JS):** Language used for browser interactivity and server-side development through Node.js.
- **Variable:** Named reference for a value.
- **DOM:** Document Object Model. Tree-like representation of HTML that JavaScript can access and modify.
- **Event:** Occurrence such as a click, key press or form submission.
- **Callback:** Function passed to another function to be called during or after an operation.
- **Promise:** Object representing the eventual result or failure of an asynchronous operation.
- **Async/await:** Syntax for writing promise-based asynchronous code.
- **Regex:** Pattern used to check or find matching text.
- **Local storage:** Browser storage that persists across normal browser restarts until cleared or removed.

Variables, conditions and loops

```
let age = 19;              // Can be reassigned
const name = "Manthan";    // Cannot be reassigned

if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}

for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

- Prefer `const` by default, and use `let` when reassignment is needed.
- `===` checks strict equality without type coercion.
- `&&` means AND, `||` means OR, and `!` means NOT.

DOM manipulation and events . M.IMP

Method/property	Purpose
`getElementById()`	Select by ID
`querySelector()`	Select first matching CSS selector
`querySelectorAll()`	Select all matching elements
`textContent`	Read or set text
`innerHTML`	Read or set HTML markup
`classList.add()`	Add CSS class
`addEventListener()`	Register event handler
`preventDefault()`	Prevent the event's default action

```
<h2 id="title">Original text</h2>
<button id="btn">Change</button>
```

```
<script>
  const title = document.querySelector("#title");

  document.querySelector("#btn")
    .addEventListener("click", () => {
      title.textContent = "Text changed!";
    });
</script>
```

Common events: `click`, `mouseover`, `mouseout`, `keydown`, `keyup`, `input`, `submit`.

Popup boxes

```
alert("Message");                  // Displays message
const name = prompt("Your name?");
const ok = confirm("Continue?");  // true or false
```

`prompt()` can return `null` when cancelled. `confirm()` returns a Boolean value.

Regex-based login form validation . M.IMP

```
const usernameRegex = /^[A-Za-z0-9_]{3,16}$/;
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const mobileRegex = /^[6-9]\d{9}$/;
const passwordRegex = /^(?=.*[A-Za-z])(?=.*\d).{8,}$/;
```

- Username: 3 to 16 letters, digits or underscores.
- Email: basic email-like format.
- Mobile: 10-digit Indian mobile format beginning with 6 to 9.
- Password: at least 8 characters, including a letter and a digit.

Pattern-checking method: Read input, apply `regex.test(value)`, show an error if invalid, and validate again on the server. Client-side checks can be bypassed.

Essential JavaScript logic programs . M.IMP

Odd or even

```
const n = 7;
console.log(n % 2 === 0 ? "Even" : "Odd");
```

Prime number

```
const n = 17;
let prime = n > 1;

for (let i = 2; i <= Math.sqrt(n); i++) {
  if (n % i === 0) {
    prime = false;
    break;
  }
}

console.log(prime ? "Prime" : "Not prime");
```

Fibonacci series

```
let a = 0, b = 1;

for (let i = 0; i < 7; i++) {
  console.log(a);
  [a, b] = [b, a + b];
}
```

Pattern printing

```
for (let i = 1; i <= 5; i++) {
  console.log("*".repeat(i));
}
```

Output:

```
*
**
***
****
*****
```

- Prime: Handle `n <= 1` separately.
- Fibonacci: Update both variables in the correct order.
- Odd/even: Use `n % 2`.
- Pattern: Outer loop controls rows; inner loop controls symbols.

Synchronous vs asynchronous

Synchronous	Asynchronous
Operations progress in sequence	An operation can start and finish later
Long-running operation can delay following work	I/O can be awaited without blocking the JavaScript thread in the same way
Simple direct flow	Often handled by callbacks, promises or `async/await`

Callback

```
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}

greet("Manthan", () => console.log("Done"));
```

Promise

```
const task = new Promise((resolve) => {
  resolve("Completed");
});
```

```
task.then(result => console.log(result));
```

Async/await

```
async function run() {
  try {
    const result = await Promise.resolve("Completed");
    console.log(result);
  } catch (error) {
    console.error(error);
  }
}

run();
```

Browser localStorage

```
localStorage.setItem("theme", "dark");
const theme = localStorage.getItem("theme");
localStorage.removeItem("theme");
```

Objects as JSON:

```
const user = { name: "Manthan", semester: 5 };

localStorage.setItem("user", JSON.stringify(user));

const savedUser = JSON.parse(localStorage.getItem("user"));
```

- `localStorage` stores strings.
- `JSON.stringify()` converts JavaScript data into JSON text.
- `JSON.parse()` converts valid JSON text back into JavaScript data.

- Do not casually store passwords, secrets or sensitive authentication tokens in localStorage.

Fetch API . Connects to CH. 4

```
async function getData() {
  try {
    const response = await fetch("/api/items");

    if (!response.ok) {
      throw new Error("HTTP " + response.status);
    }

    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}

getData();
```

Important: `fetch()` usually rejects for network failures, not merely because the server responds with `404` or `500`. Check `response.ok` or `response.status`.

Keywords: DOM, events, regex, loops, callbacks, promises, async/await, localStorage, Fetch API.

CH. 4 . APIs AND HTTP COMMUNICATION

M.IMP: Request-response mechanism, HTTP methods, headers and status codes. Also revise JSON, GET vs POST, query parameters, request body and CRUD.

Important terms

- **API:** Application Programming Interface. Defines how software components communicate.
- **HTTP:** Protocol used for web requests and responses.
- **Endpoint:** URL and method through which an API operation is accessed.
- **Header:** Metadata attached to a request or response.
- **Request body:** Data sent with a request, commonly for creating or updating a resource.
- **Query parameter:** Value appended to a URL after `?`.
- **CRUD:** Create, Read, Update and Delete.
- **JSON:** JavaScript Object Notation. Lightweight text data format.
- **XML:** Extensible Markup Language. Markup-based data format.

HTTP request-response mechanism . M.IMP

```
CLIENT / BROWSER
      |
      | Request: method, URL, headers, optional body
      v
SERVER / API
      |
      | Application logic
      v
DATABASE (if needed)
      |
      v
SERVER / API
      |
      | Response: status, headers, optional body
      v
CLIENT / BROWSER
```

HTTP methods and CRUD . M.IMP

Method	Typical purpose	CRUD mapping
`GET`	Retrieve data	Read

Method	Typical purpose	CRUD mapping
`POST`	Submit data or create a resource	Create
`PUT`	Replace a resource representation	Update
`PATCH`	Partially update a resource	Update
`DELETE`	Remove a resource	Delete

GET vs POST

GET	POST
Commonly retrieves data	Commonly submits or creates data
Parameters often in URL	Data commonly in request body
Parameters may appear in browser history/logs	Body data is not normally displayed in URL
Should not be used to change server state	Can cause server-state changes

POST is not automatically secure. Use HTTPS and suitable server-side controls.

Query parameters vs request body

```
GET /api/products?category=books&page=2
```

- `category=books` and `page=2` are query parameters.
- Useful for filters, search, sorting and pagination.

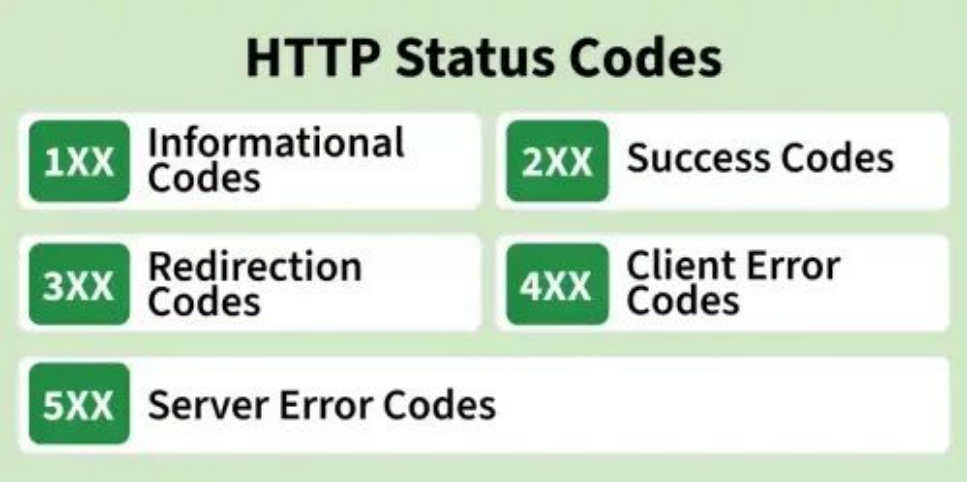
Example JSON body:

```
{
  "name": "Notebook",
  "price": 120
}
```

- Body carries submitted data, often with POST, PUT or PATCH.
- The server must parse and validate it.

HTTP status codes . M.IMP

Code	Meaning
`200 OK`	Request succeeded
`201 Created`	Resource created
`204 No Content`	Request succeeded without response content
`301` / `302`	Redirection
`400 Bad Request`	Invalid or malformed request
`401 Unauthorized`	Authentication is missing or invalid
`403 Forbidden`	Server refuses access
`404 Not Found`	Resource not found
`500 Internal Server Error`	Unexpected server failure



- 1xx: Informational.
- 2xx: Success.
- 3xx: Redirection.
- 4xx: Client error.
- 5xx: Server error.

JSON vs XML

JSON:

```
{
  "name": "Manthan",
  "semester": 5,
  "active": true
}
```

XML:

```
<student>
  <name>Manthan</name>
  <semester>5</semester>
  <active>true</active>
</student>
```

JSON	XML
Uses objects, arrays and key-value pairs	Uses opening and closing tags
Often compact and convenient for JavaScript	Explicit markup structure
Common in modern web APIs	Used in APIs and systems requiring XML

Fetch API with POST

```
async function createProduct() {
  const response = await fetch("/api/products", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      name: "Notebook",
      price: 120
    })
  });
```

```
  if (!response.ok) {
    throw new Error("Request failed: " + response.status);
  }

  return response.json();
}
```

- `method`: HTTP method.
- `headers`: Describes body format.
- `JSON.stringify()`: Converts object to JSON text.
- `body`: Sends payload.
- `response.json()`: Parses JSON response body.

Public API workflow

1. Find endpoint and API documentation.
2. Identify HTTP method and required parameters.
3. Send a request with Fetch API.
4. Check the status.
5. Parse JSON response.
6. Render data on the webpage.
7. Handle failures and loading states.

Keywords: API, HTTP, methods, headers, status codes, query parameters, request body, CRUD, JSON, XML, Fetch.

CH. 5 . BACKEND DEVELOPMENT WITH NODE.JS

NEW-SYLLABUS PRIORITY: Your PYQ report identifies this as a blind spot. It still belongs to the current syllabus.

Important terms

- **Node.js:** JavaScript runtime that executes JavaScript outside the browser.
- **npm:** Package manager and tooling ecosystem commonly used with Node.js.
- **Express.js:** Web framework for Node.js that simplifies routing and HTTP handling.
- **Route:** Mapping between a path/method and a handler.
- **Middleware:** Function that can inspect/modify a request-response and pass control onwards.
- **REST API:** API organized around resources and standard HTTP operations.
- **Authentication:** Verifying who a user is.
- **Authorization:** Determining what an authenticated user may do.
- **Database connectivity:** Connecting backend code to a database.

Node.js vs Express.js

Node.js	Express.js
JavaScript runtime	Web framework running on Node.js
Executes JavaScript outside browser	Simplifies routes, middleware and responses
Provides built-in modules/runtime APIs	Provides application-level HTTP abstractions

Setup commands

```
mkdir wad-api
cd wad-api
```

```
npm init -y
npm install express
```

- `npm init -y`: Creates default `package.json`.
- `npm install express`: Installs Express and records the dependency.

Basic Express server

```
const express = require("express");
const app = express();

app.use(express.json());

app.get("/", (req, res) => {
  res.send("WAD API is running");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

- `express()`: Creates the application.
- `express.json()`: Parses supported JSON request bodies.
- `app.get()`: Defines a GET route.
- `req`: Incoming request.
- `res`: Used to construct the response.
- `app.listen()`: Starts the server.

Run it using `node app.js`.

Routing and middleware . NEW

```
app.use((req, res, next) => {
  console.log(req.method, req.url);
  next();
});

app.get("/users/:id", (req, res) => {
  res.json({ id: req.params.id });
});
```

- `req.params.id`: Route parameter.
- `req.query`: Query parameters.
- `req.body`: Parsed request body.
- `res.send()`: Sends response.
- `res.json()`: Sends JSON.
- `next()`: Passes control to the next middleware/handler.

Middleware order matters. A middleware must end the response or call `next()` as appropriate. Otherwise, the request may remain unfinished.

REST API . CRUD skeleton

This demonstration uses a temporary in-memory array, not a persistent database.

```
const express = require("express");
const app = express();

app.use(express.json());

let items = [];
let nextId = 1;

app.get("/api/items", (req, res) => {
  res.json(items);
});
```

```
app.post("/api/items", (req, res) => {
  const item = { id: nextId++, ...req.body };
  items.push(item);
  res.status(201).json(item);
});

app.put("/api/items/:id", (req, res) => {
  const id = Number(req.params.id);
  const index = items.findIndex(item => item.id === id);

  if (index === -1) {
    return res.status(404).json({ error: "Not found" });
  }

  items[index] = { id, ...req.body };
  res.json(items[index]);
});

app.delete("/api/items/:id", (req, res) => {
  const id = Number(req.params.id);
  const exists = items.some(item => item.id === id);

  if (!exists) {
    return res.status(404).json({ error: "Not found" });
  }

  items = items.filter(item => item.id !== id);
  res.status(204).end();
});

app.listen(3000);
```

Method	Endpoint	Purpose
GET	`/api/items`	Read all items

Method	Endpoint	Purpose
POST	`/api/items`	Create item
PUT	`/api/items/:id`	Replace item
DELETE	`/api/items/:id`	Delete item

A real application must validate input and use database operations instead of relying on an in-memory array.

Database connectivity

```
Express Route
     |
     v
Validate Request
     |
     v
Database Driver / ORM
     |
     v
MongoDB or MySQL
     |
     v
Return JSON Response
```

- **MongoDB:** Document-oriented database.
- **MySQL:** Relational database.
- **Driver:** Connects application code to the database.
- Use parameterized SQL queries and safe database operations to reduce injection risks.

Authentication vs authorization

Authentication	Authorization
Verifies identity	Checks permissions
Login verification	Admin-only route
“Who are you?”	“What may you do?”

Postman API testing

1. Enter server URL.
2. Select HTTP method.
3. Add headers and request body if needed.
4. Click **Send**.
5. Inspect status code, response body and headers.

Keywords: Node.js, npm, Express, routing, middleware, REST, CRUD, database connectivity, authentication, authorization, Postman.

CH. 6 . FRONTEND DEVELOPMENT WITH REACT.JS

NEW-SYLLABUS PRIORITY: Prioritize JSX, props/state, hooks, rendering and API integration.

Important terms

- **React.js:** JavaScript library for building user interfaces.
- **Component:** Reusable UI building block.
- **JSX:** JavaScript syntax extension that resembles HTML and is commonly used in React.
- **Props:** Inputs passed from a parent to a child component.
- **State:** Component-managed data; updates can trigger a re-render.
- **Hook:** Function such as `useState()` or `useEffect()` that provides React features to function components.
- **Virtual DOM:** In-memory representation used in React's rendering/reconciliation process.

Component and JSX

```jsx
function Welcome() {
  return <h1>Hello, Manthan!</h1>;
}

export default Welcome;
```

- Component names normally start with a capital letter.
- JSX uses `className` instead of HTML's `class`.
- JavaScript expressions can be placed inside `{}`.
- JSX must follow syntax rules, including closing elements.

Props vs state

Props	State
Passed into a component	Managed by a component or hook
Parent supplies value	Updated using a state setter
Read-only from receiving component's perspective	Can change over time
Configuration/data flow	Interactive data

```jsx
function Greeting({ name }) {
  return <h2>Hello, {name}</h2>;
}

function App() {
  return <Greeting name="Manthan" />;
}
```

useState() hook

```jsx
import { useState } from "react";

function Counter() {
```

```
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
  );
}

export default Counter;
```

- `useState(0)`: Initial state is `0`.
- `count`: Current state value.
- `setCount(...)`: Requests a state update and subsequent render.
- `onClick`: Handles a click event.

useEffect() hook

Used to synchronize a component with an external system, such as fetching data or managing a connection.

```
import { useEffect } from "react";

useEffect(() => {
  console.log("Effect executed");

  return () => {
    console.log("Cleanup");
  };
}, []);
```

- An effect runs after a commit.

- An empty dependency array `[]` means the effect does not re-run because of changed reactive dependencies, although development Strict Mode may perform an extra setup/cleanup cycle.
- Cleanup runs before a relevant re-execution and when the component is removed.

Use effects for external synchronization, not as a replacement for every calculation or event handler.

Conditional and list rendering

Conditional rendering

```
function Status({ loggedIn }) {
  return loggedIn
    ? <h2>Welcome back</h2>
    : <h2>Please log in</h2>;
}
```

List rendering

```
const names = ["Asha", "Manthan", "Jay"];

function Names() {
  return (
    <ul>
      {names.map(name => (
        <li key={name}>{name}</li>
      ))}
    </ul>
  );
}
```

- Use `map()` to convert array items into UI elements.
- Provide a stable, unique `key` for each list item.

- `? :` chooses between two outcomes; `&&` can conditionally render content.

React frontend + backend API

```
import { useEffect, useState } from "react";

function Items() {
  const [items, setItems] = useState([]);
  const [error, setError] = useState("");

  useEffect(() => {
    async function loadItems() {
      try {
        const response = await fetch("/api/items");

        if (!response.ok) {
          throw new Error("Request failed");
        }

        setItems(await response.json());
      } catch (err) {
        setError(err.message);
      }
    }

    loadItems();
  }, []);

  if (error) return <p>{error}</p>;

  return (
    <ul>
      {items.map(item => (
        <li key={item.id}>{item.name}</li>
      ))}
```

```
      </ul>
    );
}

export default Items;
```

Assumes the API returns JSON objects containing unique `id` and `name` fields.

Real DOM vs Virtual DOM

Real DOM	Virtual DOM
Browser's actual document tree	In-memory representation used in React's rendering process
Actual DOM updates affect browser elements	React determines UI changes and commits updates
Directly used by browser APIs	Supports React's declarative rendering model

React is not automatically faster in every application; performance depends on application and rendering patterns.

CORS . IMP

CORS: Cross-Origin Resource Sharing.

- Browser security mechanism governing whether frontend JavaScript can read cross-origin responses.
- An origin consists of scheme, host and port.
- `http://localhost:5173` and `http://localhost:3000` are different origins.
- Server must send appropriate CORS response headers.
- Some requests trigger an `OPTIONS` preflight request.
- CORS is not a replacement for authentication or authorization.

Install:

```
npm install cors
```

Configure the API server:

```
const cors = require("cors");

app.use(cors({
  origin: "http://localhost:5173"
}));
```

Choose the actual permitted frontend origin. A wildcard origin is not valid for credentialed CORS requests; such requests require an explicit allowed origin.

Keywords: React, components, JSX, props, state, `useState`, `useEffect`, conditional rendering, lists, Fetch API, Virtual DOM, CORS.

CH. 7 . DEPLOYMENT AND MODERN WEB CONCEPTS

The syllabus includes hosting, domain management, SEO, CAPTCHA and cloud concepts. The PYQ analysis identifies this as an area with limited historical coverage.

Important terms

- **Hosting:** Making website files or application services available through a server.
- **Domain name:** Human-readable name used to access a service, such as `example.com`.
- **DNS:** Domain Name System. Maps domain names to DNS records, including records used to locate services.
- **SEO:** Search Engine Optimization. Practices that help search engines discover and understand webpage content.
- **CAPTCHA:** Challenge designed to distinguish humans from automated interactions.

- **Cloud computing:** Access to computing, storage or other IT services on demand through a network.

Domain, DNS and hosting

Domain, DNS and hosting flow

```
User enters domain
        |
        v
    DNS lookup
        |
        v
Resolved server address
        |
        v
Browser sends HTTP(S) request
        |
        v
   Web hosting/server
        |
        v
Website/application response
```

DNS resolves names. Hosting serves the application.

- Domain = human-readable address.
- DNS = resolves domain names using records.
- Hosting = infrastructure that serves the website/application.
- DNS is not the same as web hosting.

Basic deployment workflow

1. Build and test frontend and backend.
2. Configure environment variables and production settings.
3. Set up hosting or cloud services.
4. Deploy frontend and backend.
5. Configure database connectivity and allowed origins.

6. Connect domain and relevant DNS records.
7. Test routes, forms, APIs, error handling and HTTPS in production.

SEO and meta tags

```
<head>
  <title>WAD Study Guide</title>

  <meta name="description"
        content="A CH.-wise WAD study guide.">

  <meta name="viewport"
        content="width=device-width, initial-scale=1.0">

  <meta charset="UTF-8">
</head>
```

- `<title>`: Descriptive page title.
- `description`: Summary search engines may use for snippets.
- `viewport`: Supports mobile viewport layout.
- `charset`: Declares document encoding.

SEO basics: meaningful titles/headings, useful content, semantic HTML, descriptive links, mobile usability and crawlability when appropriate.

Caching clarification: HTTP response headers such as `Cache-Control` are usually used to define caching behaviour. An HTML meta description is page information, not cache expiration.

CAPTCHA

- Reduce automated form submissions and spam.
- Help distinguish legitimate user interactions from some automated activity.
- Add friction against bots.

CAPTCHA alone does not guarantee bot protection. It should complement rate limiting, validation and other safeguards.

Cloud concepts

Concept	Meaning
Compute	Processing resources that run applications
Storage	Services for storing files or data
Database service	Managed or self-managed database on infrastructure
Scalability	Ability to handle changing workload
Availability	Ability to remain accessible when needed
Deployment	Releasing an application into an environment where it can run

Keywords: Hosting, domain, DNS, SEO, meta tags, cache headers, CAPTCHA, cloud, deployment, scalability.

FINAL REVISION . WHAT TO PRIORITIZE

The priority labels follow your uploaded PYQ analysis while accounting for newer syllabus areas.

Tier 1 . Master these first

- Regex-based login form validation
- HTTP request-response, methods, headers and status codes
- CSS types and class vs ID selectors
- DOM manipulation and event handling
- Prime, Fibonacci, odd-even and pattern programs
- HTML tables using rowspan and colspan
- Client-side vs server-side scripting
- HTML form controls and attributes
- CSS positioning and Flexbox

Tier 2 . Short answers and concepts

- Callbacks, promises and synchronous vs asynchronous code
- JSON syntax and JSON vs XML
- GET vs POST and query parameters vs request body
- Popup boxes and essential DOM methods
- Bootstrap grid and responsive design
- HTTP status-code classes and common codes

Tier 3 . Do not ignore new-syllabus modules

- Node.js, Express routes and middleware
- REST API CRUD and database connectivity
- Postman, authentication and authorization
- React JSX, props and state
- useState, useEffect and list rendering
- Fetch/Axios integration and CORS
- Hosting, DNS, SEO, CAPTCHA and cloud concepts

Essential full forms

Abbreviation	Full form
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
DOM	Document Object Model
HTTP	Hypertext Transfer Protocol
API	Application Programming Interface
JSON	JavaScript Object Notation
XML	Extensible Markup Language

Abbreviation	Full form
CRUD	Create, Read, Update, Delete
REST	Representational State Transfer
JSX	JavaScript XML
CORS	Cross-Origin Resource Sharing
SEO	Search Engine Optimization
DNS	Domain Name System
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
npm	npm package manager, commonly used with Node.js

Quick technical checkpoints

- HTML structures the webpage ; CSS styles it ; JavaScript makes it interactive.
- DOM lets JavaScript access and manipulate document elements.
- HTTP methods describe the intended operation ; status codes report the outcome.
- Fetch API performs web requests ; JSON is a common data format.
- Node.js runs JavaScript outside the browser ; Express simplifies backend routing.
- React builds UI from components ; props provide inputs, while state stores changing UI data.
- CORS controls browser access to cross-origin responses, not user permissions.
- DNS resolves names ; hosting serves the application.

Study order: HTML and CSS fundamentals → JavaScript logic and DOM → HTTP and Fetch → Node.js/Express → React → deployment.

DIAGRAMS TO PRACTICE

These are simple, exam-drawable flow diagrams. They use plain text so they remain visible in Notion exports and do not depend on HTML-rendered output.

1. Client-server architecture

```
Client
  | Request
  v
Server
  | Query
  v
Database
  | Result
  v
Server
  | Response
  v
Client
```

2. Frontend-backend-database

```
Frontend
   |
   v
Backend
   |
   v
Database
```

3. Static vs dynamic

```
Static:  Browser --> HTML/CSS --> Fixed content

Dynamic: Browser --> Server --> Database
                         |
                         v
                   Generated page
```

4. DOM tree

DOM tree (simplified)

```
Document
└── html
    ├── head
    └── body
        └── div
            ├── h1
            └── button
```

The browser represents the document as a tree. JavaScript can select a node and change its content, properties or styles.

5. CSS box model

```
Margin
  └── Border
       └── Padding
            └── Content
```

6. Flexbox

```
Main axis  -------------------->

        [ A ] [ B ] [ C ]

justify-content = main-axis alignment
align-items     = cross-axis alignment
```

7. Responsive design

```
Desktop: [ A ] [ B ] [ C ] [ D ]

Mobile:  [ A ]
         [ B ]
```

```
         [ C ]
         [ D ]
```

8. Promise states

Promise state transition

```
           Pending
          /       \\
         v         v
    Fulfilled   Rejected
      (success)  (failure)
```

A settled promise is fulfilled or rejected; it does not return to pending.

9. Fetch/API flow

```
JavaScript
    |
  fetch()
    |
 HTTP request
    |
 Server / API
    |
 HTTP response
    |
 JSON parsing
    |
 JavaScript
```

10. Event handling

```
User action
    |
   Event
    |
Event listener
    |
Callback function
```

```
    |
  Action
```

11. Form validation

```
Input
  |
Regex test
  |
Valid?
 /   \
Yes   No
 |     |
Accept Error
```

MOST IMPORTANT COMPARISONS

Topic	Remember this
Client vs server	Browser vs server
Static vs dynamic	Fixed vs generated
Frontend vs backend	Visible vs behind-the-scenes
Inline vs internal vs external CSS	Element vs page vs file
Class vs ID	Reusable vs intended unique
Margin vs padding	Outside vs inside
Relative vs absolute	Normal position vs containing block
Synchronous vs async	Sequential execution vs asynchronous operation
Callback vs promise	Function passed in vs object representing eventual result
`==` vs `===`	Loose vs strict equality
`rowspan` vs `colspan`	Rows vs columns
Radio vs checkbox	One choice vs independent choices
`alert` vs `confirm` vs `prompt`	Tell vs ask vs input

Topic	Remember this
`setItem` vs `getItem`	Store vs read

30-second revision

```
HTML = Structure
CSS  = Style
JS   = Logic

Client = Browser
Server = Processing
DB     = Storage

Static  = Predefined
Dynamic = Changing/generated

rowspan = Rows
colspan = Columns

Class = .
ID    = #

Padding = Inside
Margin  = Outside

Flexbox:
justify-content = Main axis
align-items     = Cross axis

DOM = Document as a tree
Event = Browser/user occurrence
Callback = Function passed to another function

Promise:
Pending -> Fulfilled / Rejected

Fetch:
Request -> Response -> JSON

Local Storage:
set -> get -> remove -> clear
```

Final mental model

- **CH. 1:** How the web works.
- **CH. 2:** How to build and style the page.
- **CH. 3:** How to make the page interactive and dynamic.
- **CH. 4:** How applications exchange data.
- **CH. 5:** How to build the backend.
- **CH. 6:** How to build a component-based frontend.
- **CH. 7:** How to publish and operate the application.

> **Compiled BY : Manthan A. D. [in]**